



TNL-LBM: Scalable lattice Boltzmann method implementation based on Template Numerical Library

Jakub Klinkovský¹ · Pavel Eichler¹ · Robert Straka² · Tomáš Oberhuber³ · Radek Fučík³

Received: 2 September 2025 / Accepted: 26 January 2026
© The Author(s) 2026

Abstract

The lattice Boltzmann method (LBM) is a mesoscopic numerical technique widely used for solving partial differential equations such as the Navier–Stokes equations or advection–diffusion equations. It employs discrete particle distribution functions instead of macroscopic variables, offering advantages in parallelization and scalability. This paper introduces the TNL-LBM project, an advanced computational framework that implements LBM using the Template Numerical Library (TNL). TNL-LBM is designed for high-performance, large-scale simulations of turbulent flows and is available under the MIT license. The project emphasizes modularity, efficiency, and flexibility, providing a library of fundamental components and a high-level framework for developing custom solvers. Key features include a modular architecture with pluggable components, high performance through parallel algorithms and data structures provided by TNL, and scalability across multi-core CPUs, GPU accelerators, and distributed platforms. The paper outlines the components of LBM, its computational algorithm, and the design of TNL-LBM, followed by a description of input/output components and benchmark results demonstrating its performance on the Karolina supercomputer and on a compute node based on NVIDIA Tesla H100 accelerators. The performance of TNL-LBM is also compared to other open-source LBM projects.

Keywords Lattice Boltzmann method (LBM) · Computational fluid dynamics (CFD) · High-performance computing (HPC) · Scalability · Software design

Mathematics Subject Classification 65Y05 · 65Y10 · 76M28

Extended author information available on the last page of the article

Published online: 14 February 2026

1 Introduction

The lattice Boltzmann method (LBM) is a mesoscopic numerical method capable of solving a suitable class of partial differential equations such as the advection–diffusion or Navier–Stokes equations [13, 16, 23]. It uses a discrete set of particle distribution functions as main unknown variables instead of the original macroscopic variables. It has been extensively studied for the past couple of decades and has developed as an alternative numerical method to the classical finite element, finite volume, and finite difference methods. LBM was originally used for the numerical solution of the incompressible Navier–Stokes equations [35] and has been extended to various types of problems, such as fluid–solid interaction [10, 14], multiphase, multi-component and porous media flows [23], chemical and thermal transport [8, 50], radiative transport [41, 42], non-Newtonian fluid flow [11], and other problems. The method can be easily and efficiently parallelized [35], and the advent of general-purpose computing on graphics processing units (GPGPU) made large-scale and highly resolved numerical simulations of turbulent flows feasible [18, 27, 36, 47, 59, 60].

This paper describes the TNL-LBM project, an advanced computational framework implementing the lattice Boltzmann method using the Template Numerical Library (TNL) [46]. Both TNL-LBM¹ and TNL² are free and open-source projects available under the MIT license. TNL-LBM is a high-performance code designed to facilitate large-scale simulations using LBM, including problems involving turbulent flow and coupling with additional solvers. It has been used in numerous scientific and engineering applications, including environmental science, biomedical engineering, or the energy industry, where it has shown its versatility, robustness, and effectiveness. The implementation of LBM in this project was verified and validated on multiple problems in [5, 9–11, 13–16, 29]. We would like to highlight that these works include various multiphysics problems that require extensions of the lattice Boltzmann method developed within the TNL-LBM project, such as non-Newtonian fluid flow, fluid–structure interaction based on the immersed boundary method, and coupling with the mixed-hybrid finite element method (MHFEM).

The TNL-LBM project emphasizes modularity, efficiency, and flexibility. From the user's perspective, the repository contains functionality that can be used in different ways. First, there is a library of fundamental low-level components that are needed for LBM, such as collision operators, streaming schemes, data structures, and parallelization algorithms. On top of this, the TNL-LBM project provides a general high-level framework for writing LBM-based solvers and a few simple examples that show how to adapt the code for a particular problem. Using the framework is not as versatile as using the library, since it enforces certain design patterns, but it simplifies the development of custom solvers, since researchers do not need to deal with designing a solver from scratch and they can instead focus on obtaining desired results. The key features of the project include the following.

¹ <https://gitlab.com/tnl-project/tnl-lbm>

² <https://tnl-project.org/>

- *Modular architecture* with pluggable components (collision operators, streaming patterns, boundary conditions, macroscopic quantities, etc.). This modularity facilitates customization and experimentation with various LBM approaches without significant changes to the core codebase.
- *High-performance portability* achieved through the use of established data structures and parallel algorithms from the main TNL library. The project leverages TNL's advanced interface for parallel hardware architectures to ensure efficient utilization of computational resources. This includes support for common multi-core CPUs, general-purpose computing on GPU accelerators by different vendors, and computing on distributed platforms using the Message Passing Interface (MPI).
- *Scalability* is one of the primary objectives of TNL-LBM. It is designed to scale efficiently across multiple compute units within a node, as well as across multiple compute nodes. Good parallel efficiency is ensured by overlapping computation with communication and other optimization techniques. Overall, TNL-LBM enables researchers to tackle large-scale problems that require significant computational power and memory.

1.1 Related work

Several LBM implementations exist as active free and open-source projects, including Palabos [38], OpenLB [34], waLberla [3], TCLB [37], and VirtualFluids [20]. These projects share common features such as GPU acceleration, distributed computing via MPI, and various multi-physics capabilities. Beyond standard incompressible flow, Palabos, OpenLB, and waLberla can be applied to problems involving porous media, particles, free surface, thermal, or multi-phase flows. Similarly, TCLB is extended to problems with particles, thermal or multi-phase flows and VirtualFluids is an active research code with many emerging multi-physics extensions. However, the main differences of these projects lie in their design choices, programming interfaces, and implementation of LBM-specific components.

TNL-LBM distinguishes itself through its full support for distributed GPU-accelerated systems, modular architecture with pluggable components, and a permissive MIT license. The core TNL library serves as a performance portability layer for TNL-LBM, providing support for various hardware platforms without code changes in downstream projects. It also extends to multi-physics problems via models for solving thermal or non-Newtonian flows and coupling with the mixed-hybrid finite element method (MHFEM) and immersed boundary method (IBM).

Table 1 summarizes the key features of the aforementioned projects. In addition, many LBM implementations exist under non-free licenses or as inactive research projects that accompany academic publications [6, 49]. It must be noted that each software project offers its own unique and distinctive features that are not mentioned in the comparison table, such as bespoke collision operators, boundary conditions, coupling with external software projects, etc. This paper describes details related to the components implemented in the TNL-LBM project, but a comprehensive review and comparison of features provided by other projects is beyond the scope of this work. A performance comparison of the projects listed in Table 1 on a single-GPU system is presented in the last section of this paper.

Table 1 Comparison of key features among free and open-source LBM projects

Project	License	Languages	GPU Support	Scalability
TNL-LBM	MIT	C++	CUDA, HIP	128 GPUs [Sect. 6.2]
Palabos	AGPLv3	C++, Python	C++ PSTL	400 GPUs [30, 38]
OpenLB	GPLv2	C++	CUDA	512 GPUs [33]
waLBerla	GPLv3	C++, Python	CUDA, HIP	2048 GPUs [24]
TCLB	GPLv3	C++, Python, R	CUDA, HIP	48 GPUs [37]
VirtualFluids	GPLv3	C++, Python	CUDA	8 GPUs [20]

1.2 Content of the article

The paper is organized as follows. First, the components of LBM and its background are described in Sect. 2. The following Sect. 3 formulates the computational algorithm in several variants leading to a distributed and scalable implementation. Section 4 provides insights into the implementation of the TNL-LBM project and its performance optimizations. The input/output components are described in Sect. 5. The final Sect. 6 contains computational results: Firstly Sect. 6.1 presents results of numerical verification on the three-dimensional Taylor–Green vortex problem; then, Sect. 6.2 evaluates the TNL-LBM project in high-performance computing (HPC) environments such as the Karolina supercomputer [25] through several computational benchmarks, and finally Sect. 6.3 contains a performance comparison of TNL-LBM with other open-source LBM projects.

2 LBM components and background

The lattice Boltzmann method [35] (LBM) is an alternative to traditional computational methods for the solution of time-dependent PDEs such as finite difference, finite volume, and finite element methods. Based on the kinetic theory of gases, it utilizes probabilistic distributions as main degrees of freedom, with the original macroscopic variables reconstructed as raw moments. The approach based on kinetic theory makes LBM very suitable for parallel computations. However, it also poses several challenging problems such as the treatment of boundary conditions (see Sect. 2.5).

The detailed derivation of LBM is not within the scope of this article. Instead, we review the basic components related to the implementation of LBM within the presented TNL library. For further details on LBM, the reader is referred to [16, 23, 35].

2.1 Lattice and velocity set

The computational domain in TNL-LBM is a cuboid $\Omega \subset \mathbb{R}^3$ and is covered by uniform lattice $\mathcal{L}_\Omega$ defined as

$$\mathcal{L}_{\Omega} = \{\Delta x[i, j, k]^T \mid i \in \{0, \dots, N_x - 1\}, j \in \{0, \dots, N_y - 1\}, k \in \{0, \dots, N_z - 1\}\}, \quad (1)$$

where the integers N_x , N_y , and N_z determine the numbers of grid nodes along the x , y , and z axes, respectively, and Δx is the lattice spacing parameter that we set to $\Delta x = 1$ as usual in the LBM community [35]. The lattice site $\Delta x[i, j, k]^T$ corresponds to a physical point $\mathbf{x}_{i,j,k} = [i\delta_x + O_x, j\delta_y + O_y, k\delta_z + O_z]^T$, where δ_x [m] is the grid spacing parameter and $\mathbf{O} = [O_x, O_y, O_z]^T$ [m] is an offset vector that allows to fit the grid nodes to the physical coordinates of the domain. Note that "[m]" denotes the physical unit in which these quantities are measured. For further purposes, by $\mathcal{L}_{\bar{\Omega}}$ we denote the extended lattice consisting of sites used for boundary condition approximations [12].

Similarly to space discretization, the time interval $[0, t_{\max}]$ is discretized by a non-dimensional finite set $\mathcal{L}_t = \{\ell \Delta t \mid \ell \in \{0, \dots, N_t\}\}$, where N_t is the number of time steps and $\Delta t = 1$ is the non-dimensional time step. The time level $\ell \Delta t$ corresponds to the physical time $\ell \delta_t$, where $\delta_t = t_{\max}/N_t$ [s] is the time step in physical units.

LBM uses the so-called discrete velocity set $\mathcal{V}$ which comes from the discretization of the microscopic velocity space. They are typically named as DDQ , where D stands for spatial dimension, and Q denotes the number of discrete velocities per lattice site. The most common examples are D1Q3, D2Q5, D2Q9, D3Q7, D3Q15, D3Q19, and D3Q27. Not all velocity sets are suitable for accurate resolution of the desired macroscopic equation [13, 16, 35].

For the solution of the Navier–Stokes equations in $\mathbb{R}^3$, the TNL-LBM project implements the D3Q27 velocity set consisting of $Q = 27$ discrete velocities, i.e., $\mathcal{V} = \{-c, 0, c\} \times \{-c, 0, c\} \times \{-c, 0, c\}$, $c = \Delta x/\Delta t$. TNL-LBM also provides the three-dimensional velocity set D3Q7 as an alternative for solving linear advection–diffusion equations and the D2Q9 velocity set for solving the Navier–Stokes equations in $\mathbb{R}^2$. Other simpler or even more complex velocity sets may be implemented in the future.

For the purpose of lattice computations, a system of *lattice units* is defined by selecting δ_x , δ_t , and ρ_0 as conversion factors. In lattice units, we set the space step as $\Delta x = 1$, the time step as $\Delta t = 1$, and the average density $\hat{\rho}_0 = 1$. In the rest of this section, all variables are supposed in the non-dimensional form.

2.2 Discrete lattice Boltzmann equation

The basic lattice Boltzmann equation implemented in TNL-LBM is in the form of

$$f_q(\mathbf{x} + \Delta t \boldsymbol{\xi}_q, t + \Delta t) - f_q(\mathbf{x}, t) = \Delta t (C_q(\mathbf{x}, t) + S_q(\mathbf{x}, t)) \quad (2)$$

for all $q \in \{1, \dots, Q\}$, $\mathbf{x} \in \mathcal{L}_{\Omega}$, and $t \in \mathcal{L}_t$. In Equation (2), $f_q = f_q(\mathbf{x}, t)$ are the discrete density distribution functions corresponding to the microscopic velocity $\boldsymbol{\xi}_q \in \mathcal{V}$, C_q is the discrete collision operator, and S_q is the discrete source term. Note that the collision operator C_q and source term S_q depend on the entire set of Q discrete density distribution functions (i.e., the Q equations written in (2) are not

independent). Following common conventions used in the LBM community, the dependency is omitted in (2) to avoid cluttered notation.

In LBM, the moments of the discrete density distribution functions f_q are of interest, as they allow recovery of desired unknown macroscopic quantities. For instance, the scheme for the approximation of the Navier–Stokes equations is constructed such that the conserved moments correspond to the macroscopic density ρ that can be obtained in lattice units in $\mathcal{L}_\Omega \times \mathcal{L}_t$ by

$$\rho(\mathbf{x}, t) = \sum_{q=1}^Q f_q(\mathbf{x}, t) \quad (3)$$

and the macroscopic momentum density $\rho \mathbf{v}$ that can be computed in lattice units as

$$\rho(\mathbf{x}, t) \mathbf{v}(\mathbf{x}, t) = \sum_{q=1}^Q f_q(\mathbf{x}, t) \boldsymbol{\xi}_q + \frac{1}{2} \Delta t \hat{\mathbf{F}}(\mathbf{x}, t), \quad (4)$$

where $\hat{\mathbf{F}}$ denotes the external force density in lattice units exerted on the fluid. The macroscopic velocity $\mathbf{v}$ can be obtained by dividing the macroscopic momentum density $\rho \mathbf{v}$ by the macroscopic mass density ρ . Other macroscopic quantities, such as the stress tensor, can be obtained from non-conserved higher-order moments of the distribution functions [35].

2.3 Collision operator

The term $\mathcal{C}_q$ in Equation (2) denotes the discrete collision operator. In TNL-LBM, users can choose from several collision operators based on the selected discrete velocity set. The summary of collision operators implemented within the TNL-LBM project is listed in Table 2.

The operators have several parameters that drive the relaxation rates of different cumulants toward their equilibria. Their value influences the precision and overall computational time of the simulation.

Table 2 Summary of collision operators implemented in the TNL-LBM project

Collision operator	Velocity set		Reference
Single relaxation time (SRT)	D3Q7	D3Q27	[35]
Multiple relaxation time (MRT)	D3Q7	D3Q27	[18]
Improved BGK model		D3Q27	[18]
KBC model		D3Q27	[31]
Central moment-based (CLBM)	D3Q7	D3Q27	[16]
Cumulant-based (CuLBM)		D3Q27	[18]

2.4 Equilibrium function

In the kinetic theory of gases, the Maxwell–Boltzmann distribution f^{eq} describes the speeds of particles in an ideal gas that moves with a macroscopic velocity $\mathbf{v}$ in a thermodynamic equilibrium [35]. In the derivation of the discrete lattice Boltzmann equation, the equilibrium distribution function f^{eq} is discretized to obtain an approximation of the equilibrium state for the given discrete velocity set.

The form of the discrete equilibrium function is based on the solved problem and the chosen collision operator. For instance, a second-order approximation of the continuous Maxwell–Boltzmann distribution [35] and the entropic equilibrium model [31] are implemented in TNL-LBM.

2.5 Boundary conditions

The formulation of proper boundary conditions for LBM is a major complication in its applications, since the desired macroscopic conditions (e.g., given in terms of ρ and $\mathbf{v}$) must be reproduced at the mesoscopic level using the discrete density distribution functions f_q [35]. This problem does not have a unique solution, and thus multiple LBM boundary schemes exist that approximate the same macroscopic condition. The following boundary conditions are currently implemented in TNL-LBM:

- General Dirichlet boundary condition based on the equilibrium approach [35]
- Inflow boundary condition based on the equilibrium approach [35]
- Interpolated outflow boundary condition [12]
- No-slip boundary condition based on the bounce-back approach [35]
- Moment-based boundary conditions [12]
- Periodic and symmetry-plane boundary conditions [35]

Note that all boundary conditions that are currently implemented in TNL-LBM use values only from the closest layer of neighboring lattice sites. This property simplifies the implementation and interaction of boundary conditions with other components of the TNL-LBM project, such as streaming patterns and communication in distributed computing. Considering a more general class of non-local boundary conditions might limit the number of available streaming patterns.

2.6 Initialization

The initialization approach, which is used in TNL-LBM, is based on the equilibrium approach, i.e., at time $t = 0$, discrete density functions are set to their equilibria according to the supplied macroscopic density and velocity using

$$f_q(\mathbf{x}, 0) = f_q^{\text{eq}}(\rho(\mathbf{x}, 0), \mathbf{v}(\mathbf{x}, 0)) \quad (5)$$

for all lattice sites $\mathbf{x} \in \mathcal{L}_{\bar{\Omega}}$ and $q \in \{1, \dots, Q\}$. Unless noted otherwise, the values $\rho(\mathbf{x}, 0) = \hat{\rho}_0$ and $\mathbf{v}(\mathbf{x}, 0) = \mathbf{0}$ are used.

2.7 Streaming schemes

Based on Equation (2), the computational steps needed to evolve the state from the current time t to the next time level $t + \Delta t$ can be decomposed into two successive steps: the *collision* step and the *streaming* step. Depending on the order of these steps, two simple yet equivalent schemes can be formulated [35]. In the *push scheme*, the collision step is performed first and the subsequent streaming step propagates the post-collision values f_q^* , of the distribution function from the reference lattice site $\mathbf{x} \in \mathcal{L}_\Omega$ to the neighboring lattice site in the direction of the corresponding velocity $\xi_q \in \mathcal{V}$, for all $q \in \{1, \dots, Q\}$:

$$f_q^*(\mathbf{x}, t) = f_q(\mathbf{x}, t) + \Delta t (C_q(\mathbf{x}, t) + S_q(\mathbf{x}, t)), \quad (6a)$$

$$f_q(\mathbf{x} + \Delta t \xi_q, t + \Delta t) = f_q^*(\mathbf{x}, t). \quad (6b)$$

On the other hand, in the *pull scheme*, the streaming step first propagates the post-collision distribution function f_q^* to the reference lattice site $\mathbf{x} \in \mathcal{L}_\Omega$ from the neighboring lattice site in the direction of $-\xi_q$ from the previous time level, and the subsequent collision step computes the new post-collision distribution functions f_q^* at the current time level t , for all $q \in \{1, \dots, Q\}$:

$$f_q(\mathbf{x}, t) = f_q^*(\mathbf{x} - \Delta t \xi_q, t - \Delta t), \quad (7a)$$

$$f_q^*(\mathbf{x}, t) = f_q(\mathbf{x}, t) + \Delta t (C_q(\mathbf{x}, t) + S_q(\mathbf{x}, t)). \quad (7b)$$

Note that regardless of the chosen streaming scheme, the collision step is *local* in terms of the lattice sites $\mathbf{x} \in \mathcal{L}_\Omega$ (that is, collisions in different lattice sites are independent of each other), but collective for all $q \in \{1, \dots, Q\}$ since the collision operator C_q depends on the entire set of discrete density distribution functions at the given lattice site. On the other hand, the operations in the streaming step are *non-local*, but trivial as they do not require any computation (the post-collision values are merely transferred from one place to another).

Both the push and pull schemes require attention during the implementation to avoid overwriting memory locations containing values that may be needed in the future. In the former scheme, the values of f_q at the time level $t + \Delta t$ cannot be pushed to the neighboring lattice sites by overwriting the values at the current time level t , because the neighboring nodes may still be unprocessed in this time step. Similarly, after the collision step in the pull scheme, the values of f_q^* cannot overwrite the values of f_q at the reference lattice site, because those values may be pulled later when processing neighboring nodes in the same time step. Hence, the implementation of these schemes requires the use of two arrays to store the state of the simulated system. In each time step, an array A is used for input and an array B is used for output, and while going to the next time step (that is, from t to $t + \Delta t$), the arrays A and B are swapped. Hence, this approach is named *A-B pattern*. The A-B pattern with the pull scheme is implemented in the TNL-LBM project.

The *A-A pattern* [2, 58] is an alternative approach that allows to perform the streaming step in-place using just one storage array A for the density distribution functions. This feature of the streaming scheme reduces the memory requirements for a given lattice by a factor of two, which is crucial for high-resolution LBM simulations on GPU accelerators due to their limited amount of global memory. The A-A pattern is also implemented within the TNL-LBM project.

The time-stepping procedure using the A-A pattern involves different operations in even and odd iterations. In an even iteration, both pre-collision reading and post-collision writing operations happen locally at the reference lattice site $\mathbf{x} \in \mathcal{L}_\Omega$, whereas in an odd iteration, both pre-collision reading and post-collision writing operations access only neighboring lattice sites. In order to avoid read/write conflicts in the storage array, the A-A pattern involves inverting the streaming direction ξ_q at the end of even iterations and at the beginning of odd iterations.

The A-A pattern is not the only technique that facilitates in-place streaming using a single array. Alternatives include, for example, Esoteric twist [17], the compact LRnLA streaming pattern [60], Esoteric push, and Esoteric pull [39]. However, none of the aforementioned papers discuss advanced topics such as MPI communication or the implementation of boundary conditions. Hence, these streaming patterns have not yet been implemented in the TNL-LBM project.

3 Computational algorithm

The time-stepping algorithm for LBM can be generalized to cover all aforementioned streaming patterns. Algorithm 1 summarizes the steps according to our implementation on the conceptual level for the A-A pattern and the push/pull schemes with the A-B pattern.

Algorithm 1 LBM

```

1: Compute  $f_q(\mathbf{x}, 0)$  using (5) for all  $\mathbf{x} \in \mathcal{L}_{\bar{\Omega}}$  and  $q \in \{1, \dots, Q\}$       ▷ Initialization
2:  $t \leftarrow 0$ 
3: while  $t < t_{\max}$  do
4:   if specified in the model then
5:     Compute the external force density  $\hat{\mathbf{F}}(\mathbf{x}, t)$  for all  $\mathbf{x} \in \mathcal{L}_\Omega$ 
6:   end if
7:   Read values  $f_q(\mathbf{x}, t)$  for all  $\mathbf{x} \in \mathcal{L}_\Omega$  and  $q \in \{1, \dots, Q\}$       ▷ Pre-collision step
8:   Compute all macroscopic quantities ( $\rho(\mathbf{x}, t)$ ,  $\mathbf{v}(\mathbf{x}, t)$ , etc.) for all  $\mathbf{x} \in \mathcal{L}_\Omega$ 
9:   Handle boundary conditions for all relevant lattice sites in  $\mathcal{L}_{\bar{\Omega}}$ 
10:  Compute values  $f_q^*(\mathbf{x}, t)$  for all  $\mathbf{x} \in \mathcal{L}_\Omega$  and  $q \in \{1, \dots, Q\}$       ▷ Collision step
11:  Write values  $f_q(\mathbf{x}, t)$  for all  $\mathbf{x} \in \mathcal{L}_\Omega$  and  $q \in \{1, \dots, Q\}$       ▷ Post-collision step
12:  Write computed macroscopic quantities ( $\rho$ ,  $\mathbf{v}$ , etc.) to global memory
13:   $t \leftarrow t + \Delta t$ 
14: end while

```

All steps in Algorithm 1 that involve iteration over the lattice sites provide a natural way to parallelize the algorithm. While each such step could be implemented in

a separate parallel loop (e.g., launching a separate CUDA kernel), it is desirable to merge all steps in the time loop into a single parallel loop to avoid a performance penalty due to writing and reading the intermediate results. This is already indicated in Algorithm 1 which includes explicit read and write operations, and it is assumed that intermediate results are readily available for the following steps. The formulation of a parallel variant of Algorithm 1 for architectures that utilize a single shared memory, such as multicore CPUs within a single node or a single-GPU accelerator, is trivial and omitted here for brevity.

In order to utilize multiple GPUs or even multiple nodes for computing in a distributed fashion, an additional level of coarse-grained parallelism has to be introduced based on the domain decomposition approach. The global lattice $\mathcal{L}_{\bar{\Omega}}$ is divided into subdomains $\mathcal{L}_{\bar{\Omega}_j}$, $j \in \{1, \dots, N_{\text{parts}}\}$ that are processed independently using $N_{\text{ranks}} \leq N_{\text{parts}}$ independent processes (MPI ranks). Each subdomain is assigned one MPI rank that is mapped to one coarse-grain computational unit, such as a CPU core, a CPU socket, or a GPU accelerator. Each MPI rank is responsible for processing computations related to one or more subdomains that were assigned to it. In order to obtain consistent results, it must be ensured that the values of discrete density distribution functions are propagated correctly across the interfaces between subdomains. However, due to the distributed memory system, this cannot happen automatically as part of the streaming step in Algorithm 1. The solution adopted in our implementation is to extend each subdomain $\mathcal{L}_{\bar{\Omega}_j}$ with *ghost lattice sites* which are excluded from parallel processing but provide additional storage where other lattice sites will write and/or read values that will be later streamed to/from the adjacent subdomains. After the parallel loop in each time step, the data corresponding to the ghost lattice sites are exchanged via MPI among adjacent subdomains. The implementation of this procedure is explained later in Sect. 4.2.

Algorithm 2 Distributed LBM with overlapped computation and communication

```

1: for all subdomains  $\mathcal{L}_{\bar{\Omega}_j}$  in parallel do
2:   Perform initialization for all  $\mathbf{x} \in \mathcal{L}_{\bar{\Omega}_j}$  in parallel (line 1 in Algorithm 1).
3: end for
4: Exchange distribution functions  $f_q$  between adjacent subdomains.
5:  $t \leftarrow 0$ 
6: while  $t < t_{\text{max}}$  do
7:   for all subdomains  $\mathcal{L}_{\bar{\Omega}_j}$  in parallel do
8:     Start processing lattice sites next to the interfaces with other subdomains.
9:     Start processing remaining lattice sites.
10:  end for
11:  Wait until the operations started on line 8 are finished for all subdomains.
12:  Exchange distribution functions  $f_q$  between adjacent subdomains.
13:  Wait until the operations started on line 9 are finished for all subdomains.
14:   $t \leftarrow t + \Delta t$ 
15: end while

```

Furthermore, in order to ensure the scalability of the distributed LBM algorithm, it is necessary to *overlap computation with communication* to hide the extra latency

of MPI communication [26, 51]. This can be done by processing the boundary lattice sites of each subdomain separately from the interior lattice sites, copying the data between subdomains as soon as the computation on the boundary has finished, and processing the interior lattice sites independently, while all MPI ranks exchange their boundary data. This approach is summarized in Algorithm 2 using *asynchronous operations*.

The steps 8 and 9 in Algorithm 2 represent the same local operations as the steps 4 to 12 in Algorithm 1, but they are applied to different sets of lattice sites. Likewise, both steps 4 and 12 in Algorithm 2 perform the same operations; these will be explained further in Sect. 4.2.

4 Extensible design and composition of components in a simulation

From a high-level point of view, each MPI rank creates an object of the `State` class, which combines all the components needed to run a simulation.

The implementation of `State` provides all functionality needed for solving problems such as the Navier–Stokes equations. However, it is not tied to a particular problem and all problem-specific logic is implemented in separate components. The `State` class is designed to be extended via inheritance, so users can define their own subclass for a particular case and adjust the behavior of the solver by adding attributes and overriding various virtual member functions. Hence, it is possible to implement more complicated solvers for coupled problems, such as the Navier–Stokes–Fourier problem of fluid flow coupled with thermal transport [8, 50].

The class diagram of `State` is shown in the left half of Fig. 1. The `State` class has an attribute of the `LBM` class that represents one set of discrete distribution functions. Additional sets of discrete distribution functions can be represented by additional attributes in a user-defined class inheriting from `State`. Next, each `LBM` instance has a vector of `LBM_BLOCK` objects that represent the decomposition of the lattice into subdomains and additional functionality related to LBM simulations, such as variables for time-stepping or conversion between physical and lattice units. By default, each MPI rank manages just one `LBM_BLOCK`, but in general it can utilize multiple blocks that may be necessary to add for optimal load-balancing in case of non-uniform decomposition. Finally, the `LBM_BLOCK` class contains all data related to the given subdomain, which is stored in data structures that will be described later.

The implementation approach based on the extension of the `State` class is suitable for high-level adaptations that reuse various internal data structures. However, it is often necessary to modify even the low-level building blocks of the lattice Boltzmann method, such as local per-lattice-site computations in the LBM algorithm. For example, it may be desirable to switch to a different collision operator or to define additional macroscopic quantities. In order to sustain high-performance computing on GPU accelerators that have limited support for virtual functions [43], using compile-time techniques rather than dynamic polymorphism is more appropriate. Therefore, our LBM implementation is based on several template meta-programming

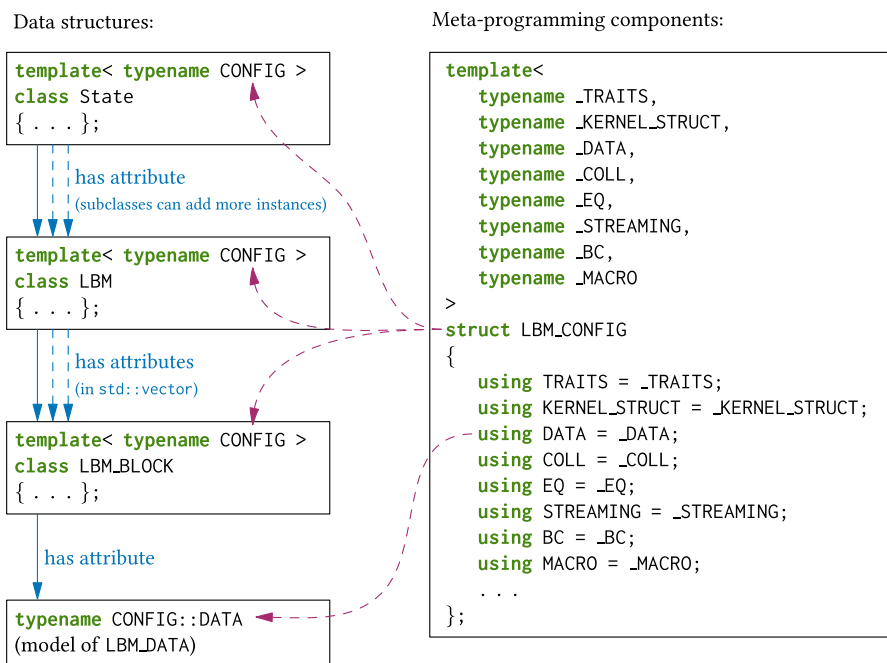


Fig. 1 Composition of the main C++ classes and meta-programming components in TNL-LBM

components that allow to configure the low-level details while preserving the generation of optimal CUDA kernels for each configuration.

The starting template meta-programming element in TNL-LBM is the `LBM_CONFIG` template class. As shown in the right half of Fig. 1, it combines the following components that are set through template parameters:

- `TRAITS` is a structure that holds the definition of fundamental types used in the solver, such as floating-point type, index type, permutations of multidimensional arrays, etc.
- `KERNEL_STRUCT` is a structure that is used in the GPU kernel to store local data related to the given lattice site. In particular, it contains a static array of Q discrete distribution functions that are read and written during the streaming steps.
- `DATA` is a structure that has the purpose of passing dynamic data and scalar parameters to the GPU kernel. It refers to the `LBM_DATA` class or its subclass, allowing users to supplement the attributes contained in `LBM_DATA` with data used by extensions of the following components.
- `COLL` is a structure that implements a particular collision operator. The action of the operator is computed in its static member function `collision` using the values in an instance of `KERNEL_STRUCT`.
- `EQ` is a structure that implements a particular discrete equilibrium function.
- `STREAMING` is a structure that implements a particular streaming scheme.

- BC is a structure that defines tags that can be assigned to lattice sites and implements actions to be taken in the CUDA kernel based on these tags. It is intended for user modification in order to specify boundary conditions for a particular simulation.
- MACRO is a structure that implements the computation of macroscopic quantities.

Only the `KERNEL_STRUCT` and `DATA` components are intended to represent data, the other components are never instantiated and serve merely to define types or algorithms in terms of static member functions. The code base provides multiple models for each component with sensible default behavior. The final composition of the type configuration can be done as follows:

```

1 using TRAITS = Traits<float>;
2 using COLL = D3Q27_CUM<TRAITS>;
3 using NSE_CONFIG = LBM_CONFIG<
4     TRAITS,                                // structure with fundamental types
5     D3Q27_KernelStruct,                    // GPU kernel structure
6     NSE_Data_XProfileInflow<TRAITS>,       // DATA structure with specific inflow BC
7     COLL,                                  // collision operator
8     typename COLL::EQ,                     // equilibrium function
9     D3Q27_STREAMING<TRAITS>,               // streaming scheme
10    D3Q27_BC_All,                           // implementation of boundary conditions
11    D3Q27_MACRO_Default<TRAITS>            // implementation of macroscopic
    quantities
12 >;

```

All the aforementioned classes `LBM_BLOCK`, `LBM`, and `State` are, in fact, templates with one parameter, `CONFIG`, as shown in Fig. 1. The main entrypoint to creating a simulation is the `State` class, or its user-defined subclass, which can be instantiated as follows:

```

1 CustomSimulationState<NSE_CONFIG> state("simulation_id", MPI_COMM_WORLD);

```

Users can subclass each model used in the `LBM_CONFIG`-type configuration and implement their modifications. However, customized components such as `BC` or `MACRO` may need to be designed carefully together with extensions of `State`, especially when custom attributes are added to the `DATA` class. More advanced examples are included in the source code of the TNL-LBM project.

4.1 Internal data structures

The TNL-LBM project relies on the Template Numerical Library (TNL) [46] for the parallel programming model and internal data structures used for the LBM algorithm. TNL is a collection of building blocks that facilitate the development of efficient numerical solvers and HPC algorithms, providing native support for modern hardware architectures such as multicore CPUs, GPUs, and distributed systems, which can be managed via a unified interface. TNL is a state-of-the-art parallel computing library comparable to, for example, Kokkos [53, 54].

Several multidimensional arrays are needed to store relevant values in the LBM implementation:

- *Distribution functions* are stored in a 4-dimensional array of size $Q \times N_x \times N_y \times N_z$. Depending on the streaming scheme, either one or two such arrays are necessary.
- *Macroscopic quantities* are stored in a 4-dimensional array of size $N_m \times N_x \times N_y \times N_z$, where N_m is the number of macroscopic quantities stored during simulation. In case of the Navier–Stokes equations, four quantities (density and three components of velocity) are stored by default, but additional user-defined quantities can be added.
- *Lattice site tags* are stored in a 3-dimensional array of size $N_x \times N_y \times N_z$ and value type `short int`. The values in this array determine the *type* of each lattice site and are used to identify different boundary conditions and geometry of immersed bodies.

The data structure used for all of these arrays is `DistributedNDArray` from TNL, which is defined as follows [52]:

```
1 template< typename NDAArray >
2 class DistributedNDArray;
```

The `DistributedNDArray` class adds functionality for distributed computing with MPI to a *local* multidimensional array, which is represented by the template parameter `NDAArray`. The `NDAArray` class in TNL is highly configurable, and its definition consists of a number of template parameters (the default types are simplified for clarity):

```
1 template< typename Value,
2           typename SizesHolder,
3           typename Permutation = /* identity by default */,
4           typename Device = TNL::Devices::Host,
5           typename Index = /* by default, the IndexType from SizesHolder */,
6           typename Overlaps = /* by default, constant 0 in each dimension */,
7           typename Allocator = /* default allocator for Value on Device */ >
8 class NDAArray;
```

The template parameters have the following role [52]:

- `Value` specifies the type of values stored in the array,
- `SizesHolder` specifies the dimension and static sizes of the array,
- `Permutation` specifies the layout of the array in memory,
- `Device` specifies the device that will be used for running operations on the array,
- `Index` specifies the type used for computing indexes in the array,
- `Overlaps` specifies the width of so-called *ghost regions* in each dimension, which are used for exchanging data in distributed computing,

- `Allocator` is the allocator type used for the allocation and deallocation of memory used by the array.

The `SizesHolder`, `Permutation`, and `Overlaps` parameters allow to specify any memory layout, including the distinction between static and dynamic sizes separately for each dimension of the array. Moreover, thanks to `Permutation`, the *logical ordering* such as $Q \times N_x \times N_y \times N_z$ for the array of distribution functions, is independent from the *data layout ordering*. For example, the aforementioned 4-dimensional array in TNL-LBM is configured with a permutation (0, 1, 3, 2), such that the index along the y -dimension changes the fastest, followed by the z -dimension and x -dimension, and the index of the distribution function $q \in \{0, 1, \dots, Q - 1\}$ changes the slowest. This data layout is advantageous for the performance of MPI communication in the case of a simple 1D domain decomposition along the x -axis (see also Sect. 4.2).

The global array corresponding to the lattice $\mathcal{L}_{\bar{\Omega}}$ must be decomposed in a *structured conforming* manner (i.e., subdomains must be orthogonal blocks arranged in a conforming grid) and the decomposition must be consistent for all the aforementioned arrays, but each rank can be assigned multiple subdomains and the rank numbering is arbitrary. All distributed data structures are collected in the `LBM_BLOCK` class that contains all data associated with one subdomain: local subarrays, subdomain sizes and offsets, indices of neighboring blocks, and ranks that own them.

To facilitate computations on GPU accelerators, two objects are needed for each array to represent data in different memory spaces. One object allocates the array data in the host memory where it can be processed by the CPU, and the other object allocates the array data in the device memory where it can be processed by the GPU accelerator. The main part of the LBM (that is, steps 8 and 9 in Algorithm 2) is computed on GPU accelerators, but CPU processing is used for initialization and output of the results. During the computation of the LBM algorithm, relevant arrays must be copied from one memory space to another whenever processing is switched from CPU to GPU or vice versa. Furthermore, the `LBM_BLOCK` class contains an instance of `LBM_DATA` (see Fig. 1), which provides access to all data that is needed from a running GPU kernel. This includes pointers to the arrays allocated in the device memory, sizes of the lattice, and scalar parameters (e.g., viscosity and parameters related to boundary conditions).

4.2 Optimization remarks

The most important optimization for distributed LBM computations is already described in Algorithm 2: Processing interior lattice sites must overlap with data synchronization on the interfaces between adjacent subdomains. This technique is implemented by launching multiple GPU kernels in different *streams* and using stream-level synchronization functions [43]. A separate stream is used for each interface with an adjacent subdomain and for the interior lattice sites. The streams for interfaces are configured with the highest priority so that the operations in these streams are executed as soon as possible. The stream for the interior is configured

with the lowest priority, as it is used for the longest-running kernel that is waited for at the end of the time step.

Data synchronization among the ranks of adjacent subdomains is governed by the `DistributedNDArraySynchronizer` class from the Template Numerical Library (TNL) [52]. A synchronizer object is created for each subdomain and each of the Q discrete distribution functions. It is responsible for synchronizing data in one subarray to and from neighbors in the direction of the corresponding vector from the velocity set. For example, when the D3Q27 velocity set is used and the lattice is distributed in a 1D manner such that each subdomain has at most two neighbors, one to the left and one to the right, 9 distribution functions (corresponding to discrete velocities with +1 in the first component) are synchronized from left to right on each interface, and different 9 distribution functions (corresponding to discrete velocities with -1 in the first component) are synchronized from right to left. Note that in case of a 2D or 3D decomposition, the communication pattern is more complicated as it is necessary to also synchronize relevant distribution functions across edges and corners of the interfaces between subdomains.

The synchronizer objects are used in Algorithm 2 in the steps 4 and 12. The synchronization procedure utilizes *pipelining* among all synchronizers to hide the latency of internal operations. Note that the implementation relies on the so-called *CUDA-aware* support [32] from the MPI library to avoid having to explicitly copy the send and receive buffers from the GPU memory to the host memory. However, unless NVIDIA GPUDirect technology [44] is functional, the MPI library may still need to internally copy data between the GPU and host memory when processing requests such as `MPI_Isend` or `MPI_Irecv`.

5 Data management and visualization

Efficient data management is essential for large-scale scientific simulations, as inadequate input/output (I/O) components can render the simulation impractical or even unusable. The TNL-LBM project addresses this challenge by using the ADIOS2 (Adaptable Input/Output System) library [22]. ADIOS2 is a cutting-edge library for scientific data management, offering a flexible framework for high-performance data I/O that is capable of processing terabytes of data per rank in exascale distributed workloads.

ADIOS2 serves two primary purposes in the TNL-LBM project: saving output data for visualization of the simulation results and checkpointing the simulation state to facilitate restarts from the last saved state.

5.1 Visualization output

The ADIOS2 library provides a generic interface that can be used to construct various data hierarchies without enforcing a specific data layout. Two notable examples of data models that utilize ADIOS2 are Fides [48] and the `vtk.xml` attribute [21]. These models enrich the dataset with metadata that maps binary data arrays to

visualization objects, enabling seamless integration with visualization tools such as ParaView [1]. An example of such visualization is shown in Fig. 2.

Compared to our previous approach of directly writing VTK files from the simulation codes, the current method retains full management of large data arrays within ADIOS2, ensuring high efficiency and scalability of the visualization pipeline. Moreover, ADIOS2 is not limited to the post-processing workflow, which involves saving output data to files on disk and processing them with external tools. By changing the *engine type* in the ADIOS2 configuration, the workflow can be adapted for *in situ* or *in-transit* visualization [40] while using the same metadata mapping attached to the ADIOS2 dataset. Exploring these advanced workflows in the TNL-LBM project is an ongoing effort.

5.2 Checkpointing

The flexibility of the ADIOS2 library in data layout makes it an ideal choice for checkpointing in the TNL-LBM project. Checkpointing involves saving the complete state of a running simulation and restoring it after a restart. This functionality allows to save the full simulation state before the allocated computation time expires and to resume the simulation from the last saved state in a new instance. Checkpointing enables splitting long simulations into several chunks that fit within the HPC system's maximum wallclock time limit, facilitating both time-constrained execution and crash recovery.

This approach results in a custom data hierarchy comprising large data arrays for all density distribution functions, boundary conditions, and scalar parameters of the simulation. Given the large memory requirements of the lattice Boltzmann method, the amount of data for checkpointing often exceeds the size of a single visualization snapshot, making the efficiency of the underlying I/O system critical.

Creating a checkpoint can be time-consuming and may run out of the allocated time limit, potentially leading to an incomplete state in the file. To prevent the corruption of the previous checkpoint, we implement a protection mechanism. The new state is written to a temporary file/directory and atomically exchanged with the previous state on disk upon successful completion. In Linux, this is achieved using the `renameat2` system call with the `RENAME_EXCHANGE` parameter [7]. This approach ensures that if the new checkpoint fails to complete, the previous checkpoint remains intact, allowing the simulation to be safely restarted from the last valid state.

6 Results

6.1 Verification

The first verification result presented in this section demonstrates the accuracy and correctness of numerical components implemented in the TNL-LBM project. We have selected the problem of a three-dimensional Taylor–Green vortex as a

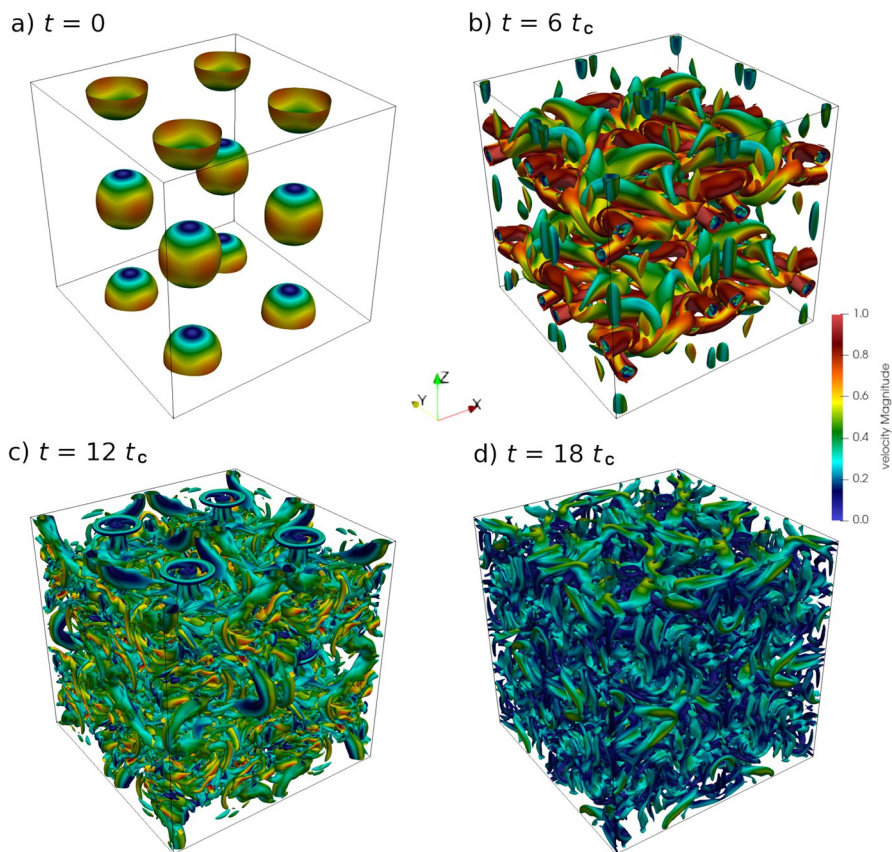


Fig. 2 Visualization of the three-dimensional Taylor–Green vortex flow using the isocontour of Q -criterion $Q_c = 1/2$ colored by velocity magnitude at four different time levels. See Sect. 6.1 for details regarding the simulation setup

canonical example of a transitional flow. The setup and parameters choice follows the specification of Problem C3.5 in [55].

The flow governed by incompressible Navier–Stokes equations is solved in the domain $\Omega = (-\pi L, \pi L) \times (-\pi L, \pi L) \times (-\pi L, \pi L)$ with periodic boundaries from the initial state given by

$$v_x = V_0 \sin\left(\frac{x}{L}\right) \cos\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right), \quad (8a)$$

$$v_y = -V_0 \cos\left(\frac{x}{L}\right) \sin\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right), \quad (8b)$$

$$v_z = 0, \quad (8c)$$

$$p = p_0 + \frac{\rho_0 V_0^2}{16} \left(\cos\left(\frac{2x}{L}\right) + \cos\left(\frac{2y}{L}\right) \right) \left(\cos\left(\frac{2z}{L}\right) + 2 \right), \quad (8d)$$

where L is the characteristic length and V_0 is the initial velocity magnitude. The initial pressure field in the lattice Boltzmann simulation is set implicitly via the density field. Additionally, the Reynolds number $\text{Re} = \frac{V_0 L}{\nu}$ is set to 1600 and the final simulation time is set to $t_{\max} = 20 t_c$, where $t_c = \frac{L}{V_0}$ is the characteristic convective time.

At the specified Reynolds number, the Taylor–Green vortex flow exhibits a transition to turbulence followed by a decay phase. Following [19], the Kolmogorov length $\eta \approx 0.01214L$ can be estimated from the enstrophy of the reference solution by [55]. As the domain size is $2\pi L$ in each direction, the space discretization should contain at least 518 nodes along each axis to resolve all turbulence scales without a sub-grid model. The reference solution by [55] was computed on a $512 \times 512 \times 512$ mesh using a pseudo-spectral method, and we choose the lattice sizes $128 \times 128 \times 128$, $256 \times 256 \times 256$, and $512 \times 512 \times 512$ for verification of the LBM implementation. Additionally, we use the D3Q27 velocity set with the cumulant collision operator and set the non-dimensional kinematic viscosity to $\nu_{LB} = 10^{-4}$ for double precision and $\nu_{LB} = 5 \cdot 10^{-4}$ for single precision.

Several mandatory results are requested by the problem specification [55]. First, we present the temporal evolution of the kinetic energy integrated on the domain Ω :

$$E_k = \frac{1}{2\rho_0|\Omega|} \int_{\Omega} \rho \mathbf{v} \cdot \mathbf{v} \, d\mathbf{x}. \quad (9)$$

The second quantity in the comparison is the temporal evolution of the kinetic energy dissipation rate $\epsilon = -\frac{dE_k}{dt}$, which we calculate numerically from E_k using a 4-point central finite difference. The dissipation rate of kinetic energy is also equal to $\epsilon = 2\nu\mathcal{E}$, where the enstrophy $\mathcal{E}$ is given by

$$\mathcal{E} = \frac{1}{2\rho_0|\Omega|} \int_{\Omega} \rho |\nabla \mathbf{v}|^2 \, d\mathbf{x}. \quad (10)$$

The dissipation rate measured by enstrophy is the third important diagnostic showing the effects of numerical diffusion of the method. These three quantities are plotted in Fig. 3 for single precision and Fig. 4 for double precision showing the convergence of lattice Boltzmann method and good agreement of the DNS-level resolution with the reference solution. Finally, Fig. 5 shows the vorticity norm on the periodic face $x = -\pi L$ at time $t = 8 t_c$ for comparison to the reference results.

6.2 Computational benchmark

The benchmark problem used in this section simulates a laminar flow (Navier–Stokes equations) in a rectangular duct for which an analytical solution can be derived [56]. The domain $\Omega = (0, L_x) \times (0, L_y) \times (0, L_z)$ (see Fig. 6) has an inflow boundary on the

left-hand side (i.e., $x = 0$), outflow boundary on the right-hand side (i.e., $x = L_x$), and solid walls with the no-slip boundary condition on the remaining sides. The inflow velocity profile is prescribed as $\mathbf{v}(0, y, z) = [v_{\text{an},x}(y, z), 0, 0]^T$, where $v_{\text{an},x}$ denotes the x -component of the velocity in the analytical solution. The kinematic viscosity is set as $\nu = 1.5 \times 10^{-5} \text{ m}^2 \text{ s}^{-1}$, and the final simulation time is $t_{\text{max}} = 100\text{s}$. The basic configuration of the TNL-LBM solver can be summarized as follows:

- D3Q27 velocity set with the cumulant collision operator,
- A-A pattern streaming scheme,
- minimal storage for macroscopic quantities (only velocity components and density),
- non-dimensional kinematic viscosity $\nu_{\text{LB}} = 10^{-3}$.

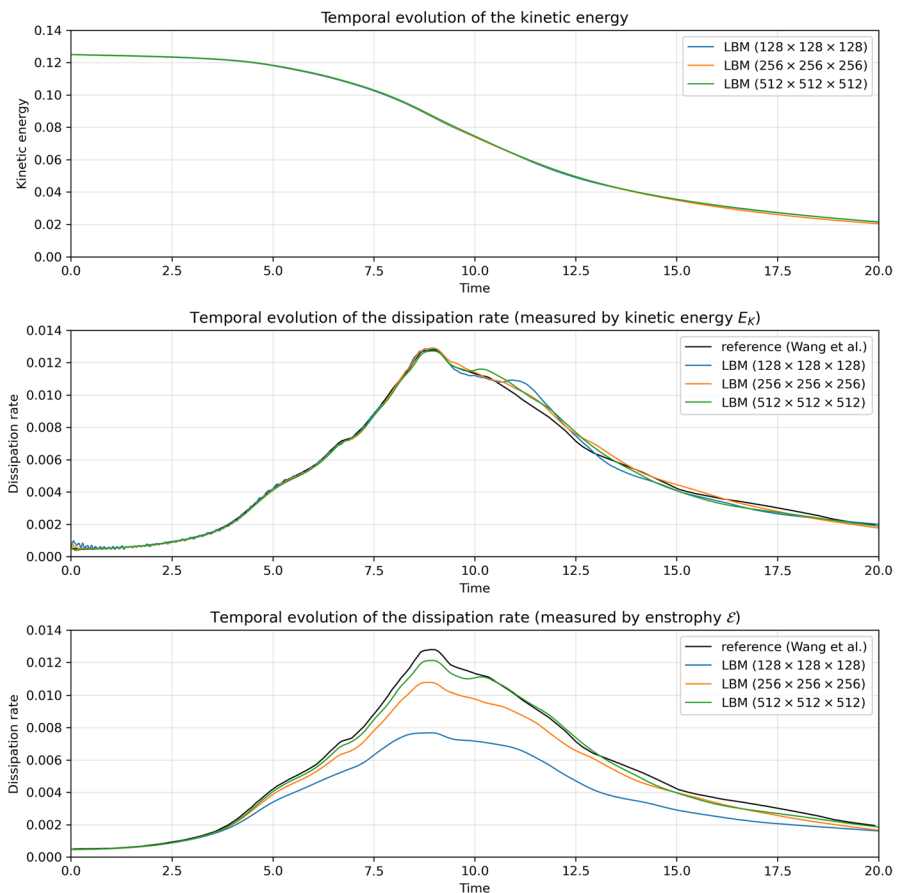


Fig. 3 Comparison of kinetic energy and dissipation rate for **LBM solutions in single precision** and reference solution from [55]

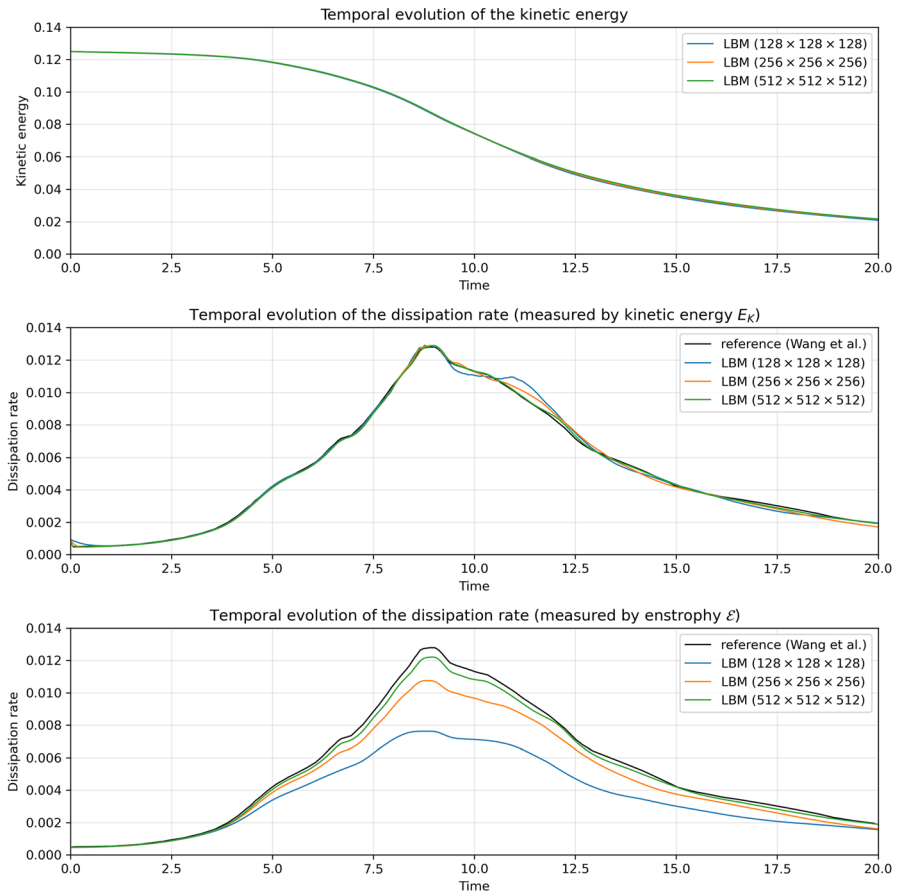


Fig. 4 Comparison of kinetic energy and dissipation rate for **LBM solutions in double precision** and reference solution from [55]

The typical metric for performance evaluation in the context of LBM is GLUPS (that is, giga-LUPS, *billions of lattice updates per second*). The value of GLUPS in a simulation is calculated by taking the number of lattice sites $N_x \times N_y \times N_z$, multiplying it by the number of completed iterations (time steps), and dividing it by the needed computational time. The values reported in this section were calculated by taking the total number of iterations in the whole simulation and the corresponding computational time (i.e., without the time spent in initialization, data output, etc.). The GLUPS values are comparable among different lattice resolutions and hardware platforms used for the computations, but they are bound to the underlying configuration of LBM. Most importantly, the results reported in this section are specific to the D3Q27 velocity set and the cumulant collision operator. However, it is interesting to note that LBM is memory-bound on modern GPU accelerators, and changing the streaming scheme in LBM (see Sect. 2.7) has negligible impact on computational performance [28].

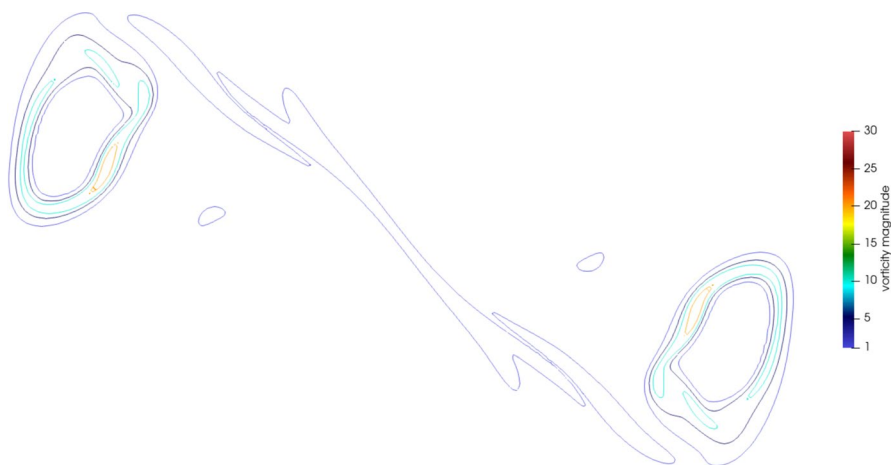


Fig. 5 Isocontours of the vorticity norm in the Taylor–Green vortex problem in one quadrant $(0, \pi L) \times (0, \pi L)$ of the periodic face $x = -\pi L$ at time $t = 8t_c$, computed on the $512 \times 512 \times 512$ lattice in double precision

For distributed computations using multiple GPUs, we define the speed-up Sp as the ratio between GLUPS using N_{ranks} and 1 GPU (assuming each rank always uses its own separate GPU), and parallel efficiency Eff , which quantifies the scalability of the computation and is defined as the speed-up divided by the number of GPUs, i.e.,

$$Eff = \frac{\text{GLUPS for } N_{\text{ranks}} \text{ GPUs}}{N_{\text{ranks}} \times (\text{GLUPS for 1 GPU})}.$$

The parallel scalability of TNL-LBM was investigated using both strong-scaling and weak-scaling studies, which are detailed in the following subsections.

6.2.1 Hardware platforms

The following results were computed on two platforms with different hardware specifications. First, the scaling performance of distributed computations using TNL-LBM is shown on the Karolina supercomputer [25], operated by IT4Innovations in Ostrava. The system comprises 72 accelerated compute nodes (see Table 3) and the number of NVIDIA A100 GPUs used in the studies ranged from 1 to 128 (i.e., 1 to 16 nodes). The solver was compiled with the `nvcc` compiler version 11.6 using the flags `-std=c++17 -O3 -DNDEBUG` and the host compiler `nvcc++` version 22.2. The communication library in this environment was OpenMPI version 4.1.2. Note that all results presented in this article were computed without utilizing the NVIDIA GPUDirect technology [44], which was not fully functional on the Karolina supercomputer at the time this study was performed. Hence, the effective bandwidth of inter-node communication was limited, because data could not be transferred from the GPU memory directly to the InfiniBand network interface and had to be buffered in the host memory instead. The maximum bandwidth observed between nodes was

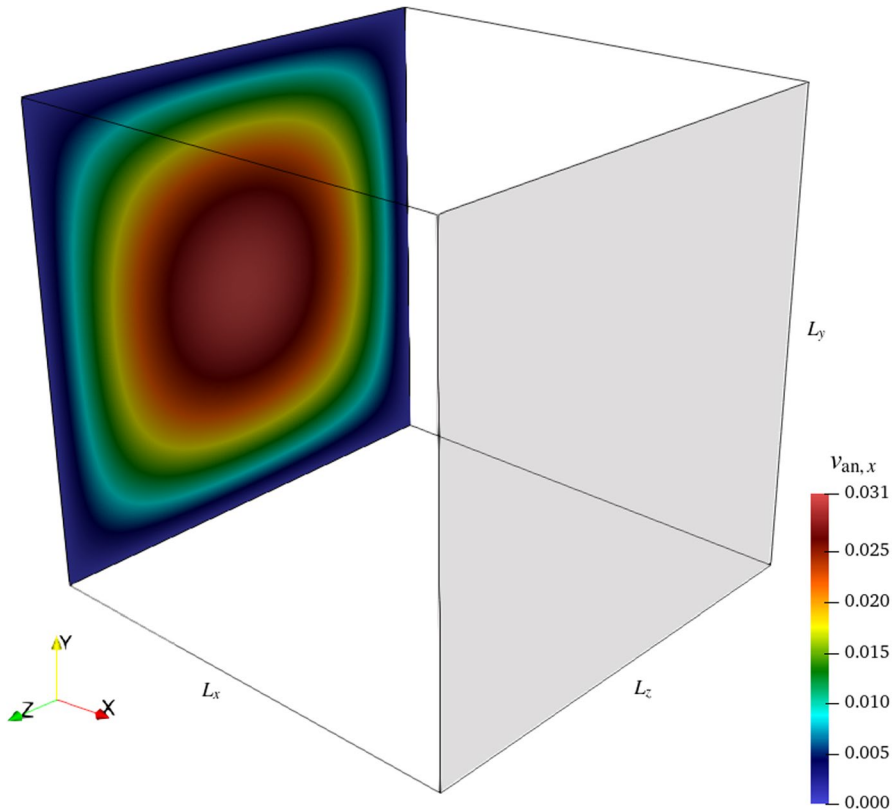


Fig. 6 Sketch of the computational domain of the benchmark problem. The color scale corresponds to the x -component of the velocity given by the analytical solution of square duct flow, and it is prescribed as the boundary condition at the inflow boundary (left back side). The opposite right front side (light gray color) is the outflow, and the remaining sides are no-slip wall boundaries

approximately 15 GB/s per direction (i.e., 30 GB/s bidirectional), which corresponds to the maximum throughput of a PCIe x16 Gen 4 bus (31.508 GB/s unidirectional [57]) rather than the aggregate throughput of InfiniBand cards. We presume that all GPUs and InfiniBand cards in the system share the same PCIe bus for the data flow with buffering in host memory.

The second system used for the computations presented here is operated by the Faculty of Mechanical Engineering of the Czech Technical University in Prague (see Table 3). It is a single compute node equipped with two 64-core AMD EPYC 9554 processors, 1.5 TB RAM, and 8 NVIDIA H100 GPU accelerators with 80 GB memory. Note that the H100 GPU has more than $2\times$ higher peak performance (TFLOPS) as well as memory bandwidth compared to the A100 (40 GB HBM2) GPU that is used in the Karolina supercomputer. All communication between the GPUs within the node is handled by the NVLink interconnection, and the inter-node network is not applicable to this system. The solver was compiled with the `nvcc`

Table 3 Hardware specifications of systems on which the benchmark was computed. The Karolina supercomputer [25] is operated by IT4Innovations (<https://www.it4i.cz/>), and the FME CTU system is operated by the Faculty of Mechanical Engineering of the Czech Technical University in Prague

	Karolina supercomputer	FME CTU system
Number of nodes	72	1
Processors per node	2	2
CPU model	AMD EPYC 7763 (64 cores)	AMD EPYC 9554 (64 cores)
Memory per node	1 TB DDR4	1.5 TB DDR4
Accelerators per node	8	8
GPU model	NVIDIA A100 (40 GB HBM2)	NVIDIA H100 (80 GB HBM3)
GPU peak performance	19.5 TFLOPS (FP32), 9.7 TFLOPS (FP64)	66.9 TFLOPS (FP32), 33.5 TFLOPS (FP64)
GPU memory bandwidth	1.55 TB/s	3.35 TB/s
Intra-node connection	NVLink 3.0 (12 sub-links, 25 GB/s per sub-link)	NVLink 4.0 (18 sub-links, 25 GB/s per sub-link)
Inter-node connection	4× 200 Gb/s InfiniBand ports	–

compiler version 12.6 using the flags `-std=c++17 -O3 -DNDEBUG` and the host compiler `g++` version 12.3.0. The communication library in this environment was OpenMPI version 4.1.5.

6.2.2 Strong-scaling study

In the strong-scaling analysis, the global lattice size remains fixed ($512 \times 512 \times 512$), while the subdomains become smaller as the number of MPI ranks increases. The corresponding problem size (16 GB in single precision, 32 GB in double precision) approaches the 40 GB memory limit of a single NVIDIA A100 GPU, restricting further scaling. The physical dimensions of the domain are $L_x = L_y = L_z = 0.25$ m.

The results from NVIDIA A100 GPUs on the Karolina supercomputer, summarized in Table 4 and Fig. 7, demonstrate near-ideal scalability up to 16 ranks (2 nodes), with efficiency dropping as the number of ranks (N_{ranks}) increases. For single precision, efficiency decreases from 0.99 at 8 ranks to 0.87 at 32 ranks and further to 0.45 at 128 ranks. Double precision maintains higher efficiency across all scales, reaching 0.64 at 128 ranks. This disparity arises because double precision computation, though slower per rank, allows better overlap with communication due to longer kernel execution times. Single-precision runs, constrained by shorter computation times, exhibit less effective hiding of communication latency, particularly at large N_{ranks} .

Communication bottlenecks emerge as subdomains shrink. Due to the 1D decomposition used in this study, the subdomains are reduced to $4 \times 512 \times 512$ elements at 128 ranks, with the communication volume per rank remaining constant (faces of subdomains). This leads to a growing communication-to-computation ratio, limiting scalability as communication eventually starts to dominate the total runtime. A multidimensional decomposition could mitigate this by distributing the communication volume more evenly, at the cost of increased latency (in 1D decomposition we

can configure the storage layout such that all communicated values are stored contiguously in memory, but in multidimensional decomposition this is not possible). However, such approach was not tested due to limited access of the authors to the Karolina supercomputer.

Table 5 presents results from a single node equipped with 8 NVIDIA H100 GPUs. Unlike the previous system, the H100 results are limited to intra-node scaling (up to 8 ranks). The single-precision performance reaches 64.6 GLUPS at 8 ranks, with a speedup of 6.6 and efficiency of 0.82. Double-precision achieves 46.5 GLUPS at 8 ranks, with a speedup of 7.9 and efficiency of 0.99. These values reflect the H100's superior per-GPU performance compared to the A100, as it enables a $2.1\times$ speedup over A100 at 8 ranks in double precision. On the other hand, the decrease in single-precision efficiency at 8 ranks indicates the need for further analysis and potential optimizations within TNL-LBM. However, the single-node limitation prevents assessing multi-node scalability, as all communication occurs via NVLink without inter-node InfiniBand transfers.

6.2.3 Weak-scaling study with extending domain size

In weak-scaling analysis, the lattice size of each subdomain remains constant, while the entire domain becomes larger as the number of MPI ranks increases. First, the domain is scaled along the x -axis, i.e., the fixed-size subdomains are stacked next to each other. Consequently, the computation-to-communication ratio is kept constant as well.

Table 6 demonstrates almost perfect weak scaling on the Karolina supercomputer for the case where the size of each subdomain is $256 \times 256 \times 256$ and the global lattice size is $(256 N_{\text{ranks}}) \times 256 \times 256$. These results are also included in Fig. 7. Our internal tests showed that the same parallel efficiency can be achieved even with larger subdomains (such as $512 \times 512 \times 512$), but not with smaller subdomains (such as $128 \times 128 \times 128$) that are presumably too small to saturate dozens of A100 GPU accelerators. Similar results are shown in Table 7 for the system equipped with H100 GPUs. Note that on this system we used a larger size of subdomains ($512 \times 512 \times 512$) because smaller sizes lead to lower efficiency in single-precision.

Table 4 Strong scaling in single and double precision on the Karolina supercomputer. The lattice size is $512 \times 512 \times 512$ in all cases. Each MPI rank was mapped to its own GPU accelerator

N_{nodes}	N_{ranks}	Single precision			Double precision		
		GLUPS	Sp	Eff	GLUPS	Sp	Eff
1	1	5.2	1.0	1.00	2.8	1.0	1.00
1	2	10.2	2.0	0.98	5.5	2.0	1.00
1	4	20.4	3.9	0.98	11.1	4.0	1.01
1	8	41.1	7.9	0.99	22.3	8.1	1.01
2	16	80.4	15.5	0.97	44.1	16.0	1.00
4	32	145.2	28.0	0.87	85.5	31.0	0.97
8	64	258.6	49.8	0.78	153.7	55.7	0.87
16	128	301.1	58.0	0.45	225.1	81.6	0.64

Table 5 Strong scaling in single and double precision on one node with NVIDIA Tesla H100 accelerators. The lattice size is $512 \times 512 \times 512$ in all cases. Each MPI rank was mapped to its own GPU accelerator

N_{ranks}	Single precision			Double precision		
	GLUPS	Sp	Eff	GLUPS	Sp	Eff
1	9.8	1.0	1.00	5.9	1.0	1.00
2	19.6	2.0	1.00	11.9	2.0	1.01
4	39.1	4.0	1.00	23.4	4.0	0.99
8	64.6	6.6	0.82	46.5	7.9	0.99

However, note that this weak-scaling analysis is not practical, since the physical size of the domain is different in each case (here $L_x = 0.25N_{\text{ranks}}$ m and $L_y = L_z = 0.25$ m). A more realistic weak-scaling study is described in the following.

6.2.4 Weak-scaling study with fixed domain size

In the second weak-scaling analysis, the primary requirement is to keep the physical size of the domain constant. In this study, we fix $L_x = L_y = L_z = 0.25$ m, the same as in the strong-scaling study. The global discrete lattice size is scaled along all three axes such that the number of nodes $N_x \times N_y \times N_z$ is (approximately) proportional to the number of ranks. The lattice size is scaled according to the formula

$$N_x = N_y = N_z = \left\lceil s \sqrt[3]{N_{\text{ranks}}} \right\rceil, \quad (11)$$

where $\lceil \cdot \rceil$ denotes rounding to the nearest integer and s is a scaling parameter selected as a large enough power-of-two that allows to fully saturate the GPUs and achieve the best weak-scaling efficiency.

For the Karolina supercomputer based on A100 GPUs, we chose $s = 512$ for single precision, giving the largest possible base lattice ($512 \times 512 \times 512$ for one rank). For double precision, we had to use a smaller base lattice with $s = 256$ due to memory limitations. The reason is that in this study, the communication size per rank is increasing (it is proportional to $N_y \times N_z$) and storage for data received from

Table 6 Weak scaling with extending domain size in single and double precision on the Karolina supercomputer. The lattice size is scaled as $256N_{\text{ranks}} \times 256 \times 256$. Each MPI rank was mapped to its own GPU accelerator

N_{nodes}	N_{ranks}	Single precision			Double precision		
		GLUPS	Sp	Eff	GLUPS	Sp	Eff
1	1	5.2	1.0	1.00	2.8	1.0	1.00
1	2	10.2	2.0	0.99	5.5	2.0	0.99
1	4	20.4	4.0	0.99	11.0	4.0	0.99
1	8	40.9	7.9	0.99	22.0	8.0	0.99
2	16	81.8	15.8	0.99	44.1	15.9	0.99
4	32	163.4	31.7	0.99	88.2	31.8	0.99
8	64	326.8	63.4	0.99	176.4	63.6	0.99
16	128	653.9	126.7	0.99	352.8	127.3	0.99

Table 7 Weak scaling with extending domain size in single and double precision on one node with NVIDIA Tesla H100 accelerators. The lattice size is scaled as $512 N_{\text{ranks}} \times 512 \times 512$. Each MPI rank was mapped to its own GPU accelerator

N_{ranks}	Single precision			Double precision		
	GLUPS	Sp	Eff	GLUPS	Sp	Eff
1	9.8	1.0	1.00	5.9	1.0	1.00
2	19.6	2.0	1.00	11.7	2.0	0.99
4	39.2	4.0	1.00	23.5	4.0	0.99
8	78.4	8.0	1.00	46.9	8.0	0.99

neighboring ranks must be allocated, so with $s = 512$ and $N_{\text{ranks}} \geq 32$ the amount of memory needed per rank would be more than the GPU accelerators have available. On the other hand, the H100 GPUs have twice as large global memory, and thus we used $s = 512$ for both single precision and double precision on the second system.

Note that the increasing communication size per rank is a limitation of the 1D domain decomposition. The TNL-LBM project also implements a general multi-dimensional decomposition; however, it has higher latency and lower throughput due to additional buffering (a 3D array cannot be ordered such that all subdomain boundaries are stored contiguously in memory, so some data have to be packed into a contiguous buffer before sending and unpacked after receiving). Therefore, in the present scenario with a cuboidal geometry, the advantage of the general decomposition over a simple 1D slicing comes only with a large number of nodes. On the other hand, the advantage of the multidimensional decomposition may be more apparent in more complex applications with a non-cuboidal geometry. Unfortunately, the benefits of the multidimensional decomposition could not be fully tested due to limited access of the authors to the Karolina supercomputer.

The results of this weak-scaling study are summarized in Table 8 and Fig. 7 for the Karolina supercomputer and in Table 9 for the compute node based on H100 GPUs. In both tables, it can be observed that Eff does not behave monotonically: For 1, 8, and 64 ranks, it is close to 1, but otherwise it is significantly lower. This effect is apparent even in single-node computations on both A100 and H100 accelerators, where all communication is accomplished through NVLink. However, this problem is not due to the communication cost—the communication is completely overlapped with computation, but the computation itself is slower than it should be. The size of the CUDA thread block is $(1, B_y, 1)$ for all lattice sizes used in this study, where $B_y = 256$ for single precision and $B_y = 128$ for double precision. It can be seen that the performance of the LBM algorithm is decreased in cases where N_y is not a multiple of B_y due to inactive threads compared to the optimal case where N_y is a multiple of B_y .

6.3 Comparison with other projects

In this section we compare the computational performance of TNL-LBM with other free and open-source LBM projects that are introduced in Table 1. Computations for this comparison were performed on a professional workstation equipped with an NVIDIA RTX A5000 graphics card (see Table 10). This system was selected

Table 8 Weak scaling with fixed domain size in single and double precision on the Karolina supercomputer. The lattice size is scaled as $N_x = N_y = N_z = \left\lfloor 512 \sqrt[3]{N_{\text{ranks}}} \right\rfloor$ in single precision and $N_x = N_y = N_z = \left\lfloor 256 \sqrt[3]{N_{\text{ranks}}} \right\rfloor$ in double precision. Each MPI rank was mapped to its own GPU accelerator

N_{nodes}	N_{ranks}	Single precision				Double precision			
		N_x	GLUPS	Sp	Eff	N_x	GLUPS	Sp	Eff
1	1	512	5.2	1.0	1.00	256	2.8	1.0	1.00
1	2	645	9.5	1.8	0.91	323	5.3	1.9	0.95
1	4	813	19.0	3.7	0.92	406	10.6	3.8	0.96
1	8	1024	40.9	7.9	0.98	512	22.4	8.1	1.01
2	16	1290	74.3	14.3	0.89	645	40.4	14.6	0.91
4	32	1625	153.8	29.6	0.93	813	82.9	30.0	0.94
8	64	2048	324.0	62.4	0.98	1024	176.2	63.7	1.00
16	128	2580	604.9	116.5	0.91	1290	301.3	108.9	0.85

for two reasons: Firstly, it is equipped with a suitable software environment where we could easily install and test all required software, and secondly, the hardware characteristics of the system allow us to study the LBM algorithm in both memory-bound and compute-bound regimes. This is because the RTX A5000 GPU features a 1:64 ratio of peak performance (FLOPS) in double precision and single precision, whereas the A100 and H100 GPUs have the ratio 1:2. As a result, the LBM algorithm is memory-bound in single precision, but becomes compute-bound in double precision, revealing the profundity of code-level optimizations implemented in the GPU kernel of each project. Unless noted otherwise, all LBM projects in this section were built with g++ 15.2.1, nvcc 13.0.88, and Python 3.13.7. All projects were configured with suitable compiler flags, including code optimization for the platform.

The configuration of the simulation for comparison was chosen from simple examples or benchmarks developed by authors of each project. We made only minor code modifications to ensure that the results are comparable. In particular, we focused on:

- switching between single precision and double precision for performance evaluation,
- setting the D3Q27 velocity set and uniform lattice size 256^3 ,

Table 9 Weak scaling with fixed domain size in single and double precision on one node with NVIDIA Tesla H100 accelerators. The lattice size is scaled as $N_x = N_y = N_z = \left\lfloor 512 \sqrt[3]{N_{\text{ranks}}} \right\rfloor$. Each MPI rank was mapped to its own GPU accelerator

N_{ranks}	N_x	Single precision			Double precision		
		GLUPS	Sp	Eff	GLUPS	Sp	Eff
1	512	9.8	1.0	1.00	5.9	1.0	1.00
2	645	15.6	1.6	0.79	10.0	1.7	0.84
4	813	33.2	3.4	0.85	20.8	3.5	0.88
8	1024	79.7	8.1	1.02	48.2	8.2	1.02

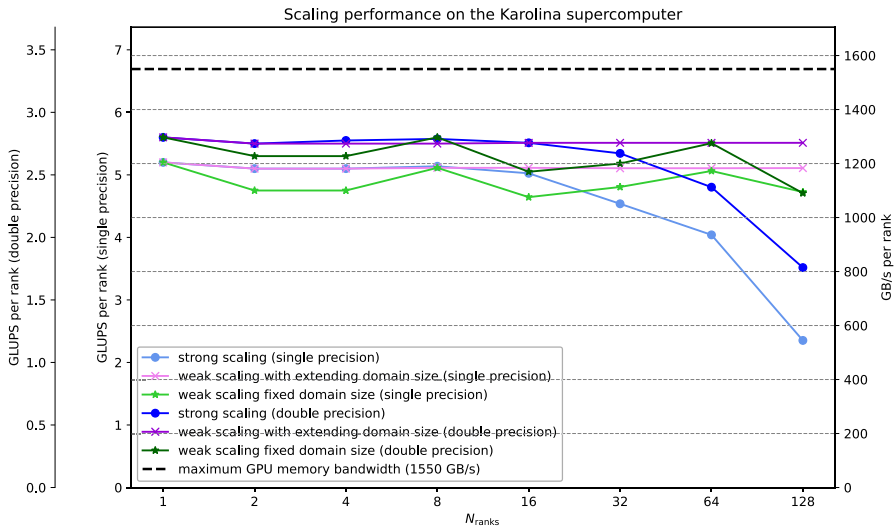


Fig. 7 Comparison of GLUPS per rank and achieved memory bandwidth on the Karolina supercomputer across all scaling scenarios. Values are based on Table 4, 6 and 8. The memory bandwidth is calculated by multiplying GLUPS by the number of values per lattice site (58) that are read/written in the global memory and by the data type size in bytes (4 for single precision and 8 for double precision). The values on each curve are also proportional to the parallel efficiency (η_p)

- using the facilities provided by each project to set the cumulant collision operator [18], the state of the art, and one of the most complex options for fluid flow simulations to date (only OpenLB does not have a functioning cumulant collision operator, so we used the OpenLB's default BGK operator),
- ensuring that only "simple" boundary conditions are used in the simulation.

We resorted to setting up either a *channel flow* or a *lid-driven cavity* case, depending on what is available in each project (see Table 11). Both cases involve only the bounce-back boundary condition for no-slip walls, a simple condition to prescribe velocity, and a simple outflow condition in case of the channel flow. While the implementation of these conditions and the choice of simulation parameters are different across all projects, they have negligible impact on performance differences.

The results in Table 11 show that waLBerla delivers the highest performance both in single precision (3.12 GLUPS) and double precision (0.656 GLUPS). The waLBerla project uses a hybrid approach for problem definition: The simulation developer writes a high-level skeleton code in C++ and detailed problem specification in Python, which is passed to a code generator producing an optimized C++ code and GPU kernels adapted for the given problem [4]. Compiling the generated code along with the developer's skeleton and waLBerla's API results in a highly optimized binary code while providing high modularity and flexibility.

TNL-LBM seized the second place based on single precision performance (2.892 GLUPS), but its performance in double precision (0.354 GLUPS) is significantly lower than waLBerla's. The collision operator in TNL-LBM is hand-written based

Table 10 Hardware specifications of the system used for the performance comparison of open-source LBM projects

Processors per node	1
CPU model	Intel Core i9-10900X (10 cores)
Memory per node	256 GB DDR4
Accelerators per node	1
GPU model	NVIDIA RTX A5000 (24 GB GDDR6)
GPU peak performance in FP32	22.7 TFLOPS
GPU peak performance in FP64	0.43 TFLOPS
GPU memory bandwidth	768 GB/s

on the publication [18], and the implementation does not employ advanced code optimization techniques that are used by waLBerla. The results show that relying on compiler optimizations is sufficient in single precision where the algorithm is memory-bound, but the project needs to implement code optimizations to improve performance in the compute-bound regime.

The VirtualFluids project provides a reference implementation of the cumulant collision operator, which was independently implemented by other projects based on the publication [18]. The performance of VirtualFluids reaches competitive 2.708 GLUPS in single precision, but shows the highest computational complexity with only 0.167 GLUPS in double precision.

OpenLB is the only project in the comparison with a different collision operator. Nevertheless, it comprises the same amount of read/write operations due to the D3Q27 velocity set, so its performance of 2.509 GLUPS in single precision, where the LBM algorithm is memory-bound, is comparable to other projects. But the BGK operator has lower computational complexity than the cumulant operator, so the performance in double precision (0.411 GLUPS) is overrated and not comparable to other projects.

Palabos shares a common history and philosophy with OpenLB [38], sustaining many similarities to date. Unlike OpenLB, Palabos provides the implementation of the cumulant collision operator. However, Palabos has support for GPU computing only in a pre-release, not in the public GitLab repository. The GPU acceleration is based on parallel STL algorithms (stdpar) [45], whereas all other projects use

Table 11 Performance comparison of open-source LBM projects on NVIDIA RTX A5000 (lattice size 256^3 , velocity set D3Q27)

Project	Case	Collision	Single precision	Double precision
waLBerla	Lid-driven cavity	Cumulant	3.120 GLUPS	0.656 GLUPS
TNL-LBM	Channel flow	Cumulant	2.892 GLUPS	0.354 GLUPS
VirtualFluids	Lid-driven cavity	Cumulant	2.708 GLUPS	0.167 GLUPS
OpenLB	Lid-driven cavity	BGK	2.509 GLUPS	0.411 GLUPS
Palabos	Lid-driven cavity	Cumulant	0.197 GLUPS	0.191 GLUPS
TCLB	Channel flow	Cumulant	0.680 GLUPS	0.283 GLUPS

CUDA [43] directly. The Palabos application used in this comparison was built with NVIDIA's compiler `nvc++` from NVHPC 25.9 [45].

Similarly to waLberla, TCLB employs another high-level programming language to generate the CUDA code for the GPU kernel. However, it is R rather than Python and the code optimization level is not on par with waLberla, resulting in 0.68 GLUPS in single precision and 0.283 GLUPS in double precision. Nevertheless, TCLB outperforms Palabos in both precisions and VirtualFluids in double precision.

Overall, the results in Table 11 show balanced performance in single precision across most of the compared projects. This indicates that in the memory-bound regime, it is possible to achieve competitive performance even without advanced code optimization techniques applied to the numerical scheme. However, large differences in double precision performance indicate that the implementation of the cumulant collision operator is very different in each project and code optimization techniques can improve the performance by multiple factors. It is also interesting to note that while the RTX A5000 GPU has a 1:64 ratio of peak performance in double precision and single precision, the ratio of GLUPS performance achieved by waLberla is approximately 1:5.

7 Conclusions

The TNL-LBM project represents a robust and versatile implementation of the lattice Boltzmann method (LBM), designed to address the challenges of large-scale, high-fidelity simulations in computational fluid dynamics (CFD) and multiphysics applications. By leveraging the Template Numerical Library (TNL), the framework achieves a balance between modularity, performance, and scalability, enabling researchers to tackle complex problems ranging from turbulent flow modeling to fluid–structure interaction and coupled multiphysics systems. The modular architecture of TNL-LBM allows seamless integration of diverse LBM variants, boundary conditions, and collision operators, while its high-performance design ensures efficient utilization of modern heterogeneous computing architectures, including multi-core CPUs, GPUs, and distributed memory systems.

The computational algorithm, formulated to support both single-node and distributed implementations, demonstrates strong scalability on the Karolina supercomputer and NVIDIA H100 GPUs. The benchmark results highlight the ability of the framework to maintain high parallel efficiency even under challenging scaling scenarios, with double-precision computations showing superior scalability compared to single precision due to more efficient utilization of the hardware. These findings underscore the importance of code optimizations for data layout and communication, such as aligning memory access patterns and minimizing latency. While the presented benchmarks include only results obtained on NVIDIA GPUs, note that thanks to the performance portability layer provided by TNL the project also supports running simulations on the AMD platform via the ROCm/HIP toolkit. The performance comparison of free and open-source LBM projects demonstrates competitiveness and high performance of the TNL-LBM project.

Beyond technical performance, the use of the MIT license further enhances the value of the project to the scientific community, fostering reproducibility, collaboration, and

integration into proprietary workflows. This licensing choice, combined with demonstrated computational scalability, lowers barriers to adoption and encourages collaboration between academia and industry. The validation of the framework on a wide range of problems, including non-Newtonian flows and coupled solvers, demonstrates its reliability and adaptability to real-world engineering and scientific challenges. Future plans for the project currently include integrating advanced collision models for compressible flows, improving the implementation of the immersed boundary method for fluid–structure interaction, and utilizing advanced strategies for *in situ/in-transit* visualization.

In summary, TNL-LBM provides a powerful and flexible tool for high-performance LBM simulations that combine algorithmic innovation with modern software engineering practices. Its success in scaling to 128 of GPU-accelerated ranks on Karolina and achieving high performance on H100 systems highlights the project’s capabilities in performing large-scale and high-fidelity simulations where accuracy and efficiency are crucial factors.

Acknowledgements This work was supported by the projects LUAUS25049 and e-INFRA CZ (ID:90254) of the Ministry of Education, Youth and Sports of the Czech Republic and by the project 26-21483S of the Czech Science Foundation. R.S. was supported by the by statutory activities – work no. 16.16.110.663.

Author contributions J.K. and P.E. wrote the main manuscript text. All authors contributed to the software project described in the manuscript text. J.K. performed the computational studies. All authors reviewed the manuscript.

Funding Open access publishing supported by the institutions participating in the CzechELib Transformative Agreement.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ahrens J, Geveci B, Law C (2005) ParaView: an end-user tool for large-data visualization. Butterworth-Heinemann Burlington chap 36:717–731. <https://doi.org/10.1016/b978-012387582-2/50038-1>
2. Bailey P, Myre J, Walsh SDC, et al (2009) Accelerating lattice Boltzmann fluid flow simulations using graphics processors. In: 2009 International Conference on Parallel Processing. IEEE, pp 550–557, <https://doi.org/10.1109/ICPP.2009.38>
3. Bauer M, Eibl S, Godenschwager C et al (2021) waLBerla: a block-structured high-performance framework for multiphysics simulations. Comput & Math Appl 81:478–501. <https://doi.org/10.1016/j.camwa.2020.01.007>

4. Bauer M, Köstler H, Rüdé U (2021) lbmpy: automatic code generation for efficient parallel lattice Boltzmann methods. *J Comput Sci* 49:101269. <https://doi.org/10.1016/j.jocs.2020.101269>
5. Beneš M, Eichler P, Fučík R et al (2022) Experimental and numerical investigation of air flow through the distributor plate in a laboratory-scale model of a bubbling fluidized bed boiler. *Jpn J Ind Appl Math* 39(3):943–958. <https://doi.org/10.1007/s13160-022-00518-x>
6. Chen Y, Valocchi AJ, Kang Q et al (2019) Inertial effects during the process of supercritical CO₂ displacing brine in a sandstone: lattice Boltzmann simulations based on the continuum-surface-force and geometrical wetting models. *Water Resour Res* 55(12):11144–11165. <https://doi.org/10.1029/2019WR025746>
7. Corbet J (2013) Exchanging two files. <https://lwn.net/Articles/569134/>, [Accessed April 2025]
8. Dapelo D, Simonis S, Krause MJ et al (2021) Lattice-Boltzmann coupled models for advection-diffusion flow on a wide range of Péclet numbers. *J Comput Sci* 51:101363. <https://doi.org/10.1016/j.jocs.2021.101363>
9. Eichler P, Fuka V, Fučík R (2021) Cumulant lattice Boltzmann simulations of turbulent flow above rough surfaces. *Comput & Math Appl* 92:37–47. <https://doi.org/10.1016/j.camwa.2021.03.016>
10. Eichler P, Fučík R, Straka R (2021) Computational study of immersed boundary-lattice Boltzmann method for fluid-structure interaction. *Discret & Contin Dynam Syst-S* 14(3):819. <https://doi.org/10.3934/dcdss.2020349>
11. Eichler P, Galabov R, Fučík R et al (2023) Non-Newtonian turbulent flow through aortic phantom: experimental and computational study using magnetic resonance imaging and lattice Boltzmann method. *Comput & Math Appl* 136:80–94. <https://doi.org/10.1016/j.camwa.2023.01.031>
12. Eichler P, Fučík R, Strachota P (2024) Investigation of mesoscopic boundary conditions for lattice Boltzmann method in laminar flow problems. *Comput & Math Appl* 173:87–101. <https://doi.org/10.1016/j.camwa.2024.08.009>
13. Fučík R, Straka R (2021) Equivalent finite difference and partial differential equations for the lattice Boltzmann method. *Comput & Math Appl* 90:96–103. <https://doi.org/10.1016/j.camwa.2021.03.014>
14. Fučík R, Eichler P, Straka R et al (2019) On optimal node spacing for immersed boundary-lattice Boltzmann method in 2D and 3D. *Comput & Math Appl* 77(4):1144–1162. <https://doi.org/10.1016/j.camwa.2018.10.045>
15. Fučík R, Galabov R, Pauš P et al (2020) Investigation of phase-contrast magnetic resonance imaging underestimation of turbulent flow through the aortic valve phantom: experimental and computational study using lattice Boltzmann method. *Magn Reson Mater Phys, Biol Med* 33(5):649–662. <https://doi.org/10.1007/s10334-020-00837-5>
16. Fučík R, Eichler P, Klínek J et al (2022) Lattice Boltzmann method analysis tool (LBMAT). *Numer Algorithms*. <https://doi.org/10.1007/s11075-022-01476-8>
17. Geier M, Schönherr M (2017) Esoteric twist: an efficient in-place streaming algorithmus for the lattice Boltzmann method on massively parallel hardware. *Computation* 5(2):19. <https://doi.org/10.3390/computation5020019>
18. Geier M, Schönherr M, Pasquali A et al (2015) The cumulant lattice Boltzmann equation in three dimensions: theory and validation. *Comput & Math Appl* 70(4):507–547. <https://doi.org/10.1016/j.camwa.2015.05.001>
19. Geier M, Lenz S, Schönherr M et al (2021) Under-resolved and large eddy simulations of a decaying Taylor-Green vortex with the cumulant lattice Boltzmann method. *Theoret Comput Fluid Dyn* 35(2):169–208. <https://doi.org/10.1007/s00162-020-00555-7>
20. Geier M, Kutscher K, Schönherr M et al (2025) VirtualFluids open source parallel LBM solver. *Comput Phys Commun* 317:109810. <https://doi.org/10.1016/j.cpc.2025.109810>
21. Godoy WF (2020) ADIOS2 Documentation visualizing data. Oak Ridge National Laboratory, <https://adios2.readthedocs.io/en/v2.10.2/ecosystem/visualization.html>
22. Godoy WF, Podhorski N, Wang R, et al (2020) ADIOS 2: The adaptable input output system. a framework for high-performance data management. *SoftwareX* 12:100561. <https://doi.org/10.1016/j.softx.2020.100561>
23. Guo Z, Shu C (2013) Lattice Boltzmann method and its application in engineering, vol 3. *World Sci*. <https://doi.org/10.1142/8806>
24. Holzer M, Bauer M, Köstler H et al (2021) Highly efficient lattice Boltzmann multiphase simulations of immiscible fluids at high-density ratios on CPUs and GPUs through code generation. *Int J High Perform Comput Appl* 35(4):413–427. <https://doi.org/10.1177/10943420211016525>
25. IT4Innovations (2025) It4innovations documentation – Karolina cluster. <https://docs.it4i.cz/karolina/introduction/>, [Accessed April 2025]

26. Januszewski M, Kostur M (2014) Sailfish: a flexible multi-GPU implementation of the lattice Boltzmann method. *Comput Phys Commun* 185(9):2350–2368. <https://doi.org/10.1016/j.cpc.2014.04.018>
27. Kang SK, Hassan YA (2013) The effect of lattice models within the lattice Boltzmann method in the simulation of wall-bounded turbulent flows. *J Comput Phys* 232(1):100–117. <https://doi.org/10.1016/j.jcp.2012.07.023>
28. Klinkovský J (2023) Data structures and parallel algorithms for numerical solvers in computational fluid dynamics. PhD thesis, Czech Technical University in Prague, Faculty of Nuclear Sciences and Physical Engineering, Prague, Czech Republic, <https://hdl.handle.net/10467/112277>.
29. Klinkovský J, Trautz AC, Fučík R et al (2023) Lattice Boltzmann method-based efficient GPU simulator for vapor transport in the boundary layer over a moist soil: development and experimental validation. *Comput & Math Appl* 138:65–87. <https://doi.org/10.1016/j.camwa.2023.02.021>
30. Kotsalos C, Latt J, Beny J et al (2021) Digital blood in massively parallel CPU/GPU systems for the study of platelet transport. *Interface Focus* 11(1):20190116. <https://doi.org/10.1098/rsfs.2019.0116>
31. Krämer A, Wilde D, Küllmer K et al (2019) Pseudoentropic derivation of the regularized lattice Boltzmann method. *Phys Rev E* 100(2):023302. <https://doi.org/10.1103/PhysRevE.100.023302>
32. Kraus J (2013) An introduction to CUDA-aware MPI. <https://developer.nvidia.com/blog/introduction-on-cuda-aware-mpi/>, NVIDIA Developer Blog, [Accessed April 2025]
33. Krause M, Kummerländer A, Frank M, et al (2022) Performance on HoreKa using OpenLB 1.5. <https://www.openlb.net/performance/>, [online, OpenLB documentation website]
34. Krause MJ, Kummerländer A, Avis SJ et al (2021) OpenLB open source lattice Boltzmann code. *Comput & Math Appl* 81:258–288. <https://doi.org/10.1016/j.camwa.2020.04.033>
35. Krüger T, Kusumaatmaja H, Kuzmin A et al (2017) The lattice Boltzmann method. Springer. <https://doi.org/10.1007/978-3-319-44649-3>
36. Kumar P, Kutscher K, Mößner M et al (2018) Validation of a VRANS-model for turbulent flow over a porous flat plate by cumulant lattice Boltzmann DNS/LES and experiments. *J Porous Media*. <https://doi.org/10.1615/JPorMedia.v21.i5.60>
37. Łaniewski-WoŃk Ł, Rokicki J (2016) Adjoint lattice Boltzmann for topology optimization on multi-GPU architecture. *Comput & Math Appl* 71(3):833–848. <https://doi.org/10.1016/j.camwa.2015.12.043>
38. Latt J, Malaspinas O, Kontaxakis D et al (2020) Palabos: parallel lattice Boltzmann solver. *Comput & Math Appl*. <https://doi.org/10.1016/j.camwa.2020.03.022>
39. Lehmann M (2022) Esoteric pull and esoteric push: two simple in-place streaming schemes for the lattice Boltzmann method on GPUs. *Computation*. <https://doi.org/10.3390/computation10060092>
40. Mazen F, Givord L, Gueunet C (2023) Catalyst-ADIOS2: In transit analysis for numerical simulations using Catalyst 2 API. In: Bienz A, Weiland M, Baboulin M et al (eds) *High Performance Computing*. Springer Nature Switzerland, Cham, pp 269–276
41. Mink A, McHardy C, Bressel L et al (2020) Radiative transfer lattice Boltzmann methods: 3D models and their performance in different regimes of radiative transfer. *J Quant Spectrosc Radiat Transfer* 243:106810. <https://doi.org/10.1016/j.jqsrt.2019.106810>
42. Mink A, Schediwy K, Posten C et al (2022) Comprehensive computational model for coupled fluid flow, mass transfer, and light supply in tubular photobioreactors equipped with glass sponges. *Energies*. <https://doi.org/10.3390/en15207671>
43. NVIDIA (2025a) CUDA toolkit documentation, version 12.8. <https://docs.nvidia.com/cuda/archive/12.8.0/>
44. NVIDIA (2025b) Developing a Linux kernel module using GPUDirect RDMA. <https://docs.nvidia.com/cuda/gpudirect-rdma/index.html>
45. NVIDIA (2025c) NVIDIA HPC Compilers User's Guide version 25.9. <https://docs.nvidia.com/hpc-sdk/compilers/hpc-compilers-user-guide/index.html>
46. Oberhuber T, Klinkovský J, Fučík R (2021) TNL: Numerical library for modern parallel architectures. *Acta Polytechnica* 61(SI):122–134. <https://doi.org/10.14311/AP.2021.61.0122>
47. Peng C, Geneva N, Guo Z et al (2018) Direct numerical simulation of turbulent pipe flow using the lattice Boltzmann method. *J Comput Phys* 357:16–42. <https://doi.org/10.1016/j.jcp.2017.11.040>
48. Ross C, Pugmire D, Geveci B, et al (2023) Fides User Guide Version 1.2.0. Kitware, <https://fides.readthedocs.io/en/v1.2.0/>
49. Santos JE, Gigliotti A, Bihani A et al (2022) MPLBM-UT: multiphase LBM library for permeable media analysis. *SoftwareX* 18:101097. <https://doi.org/10.1016/j.softx.2022.101097>
50. Sharma KV, Straka R, Tavares FW (2020) Current status of lattice Boltzmann methods applied to aerodynamic, aeroacoustic, and thermal flows. *Prog Aerosp Sci* 115:37. <https://doi.org/10.1016/j.paerosci.2020.100616>

51. Succi S, Amati G, Bernaschi M et al (2019) Towards exascale lattice Boltzmann computing. *Comput & Fluids* 181:107–115. <https://doi.org/10.1016/j.compfluid.2019.01.005>
52. TNL project developers (2025) Template Numerical Library documentation. <https://tnl-project.org/documentation/>
53. Trott C, Berger-Vergiat L, Poliakoff D et al (2021) The Kokkos ecosystem: comprehensive performance portability for high performance computing. *Comput Sci Eng* 23(5):10–18. <https://doi.org/10.1109/MCSE.2021.3098509>
54. Trott CR, Lebrun-Grandi D, Arndt D et al (2022) Kokkos 3: programming model extensions for the exascale era. *IEEE Trans Parallel Distrib Syst* 33(4):805–817. <https://doi.org/10.1109/TPDS.2021.3097283>
55. Wang ZJ, Fidkowski K, Abgrall R et al (2013) High-order CFD methods: current status and perspective. *Int J Numer Meth Fluids* 72(8):811–845. <https://doi.org/10.1002/fld.3767>
56. White FM (2005) *Viscous fluid flow*. McGraw-Hill, New York
57. Wikipedia contributors (2025) *PCI Express* — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=PCI_Express&oldid=1315994286, [Online; accessed 9-October-2025]
58. Wittmann M, Zeiser T, Hager G et al (2013) Comparison of different propagation steps for lattice Boltzmann methods. *Comput & Math Appl* 65(6):924–935. <https://doi.org/10.1016/j.camwa.2012.05.002>
59. Wittmann M, Zeiser T, Hager G et al (2013) Domain decomposition and locality optimization for large-scale lattice Boltzmann simulations. *Computers & Fluids* 80:283–289. <https://doi.org/10.1016/j.compfluid.2012.02.007>
60. Zakirov A, Perepelkina A, Levchenko V et al (2021) Streaming techniques: revealing the natural concurrency of the lattice Boltzmann method. *J Supercomput* 77(10):11911–11929. <https://doi.org/10.1007/s11227-021-03762-z>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Jakub Klinkovský¹ · Pavel Eichler¹ · Robert Straka² · Tomáš Oberhuber³ · Radek Fučík³

✉ Jakub Klinkovský
jakub.klinkovsky@jfifi.cvut.cz

Pavel Eichler
pavel.eichler@jfifi.cvut.cz

Robert Straka
strakrob@agh.edu.pl

Tomáš Oberhuber
tomas.oberhuber@jfifi.cvut.cz

Radek Fučík
radek.fucik@jfifi.cvut.cz

¹ Department of Software Engineering, Faculty of Nuclear Sciences and Physical Engineering, Czech Technical University in Prague, Trojanova 13, 12000 Prague, Czech Republic

² AGH University of Science and Technology, Adama Mickiewicza 30, 30-059 Krakow, Poland

³ Department of Mathematics, Faculty of Nuclear Sciences and Physical Engineering, Czech Technical University in Prague, Trojanova 13, 12000 Prague, Czech Republic